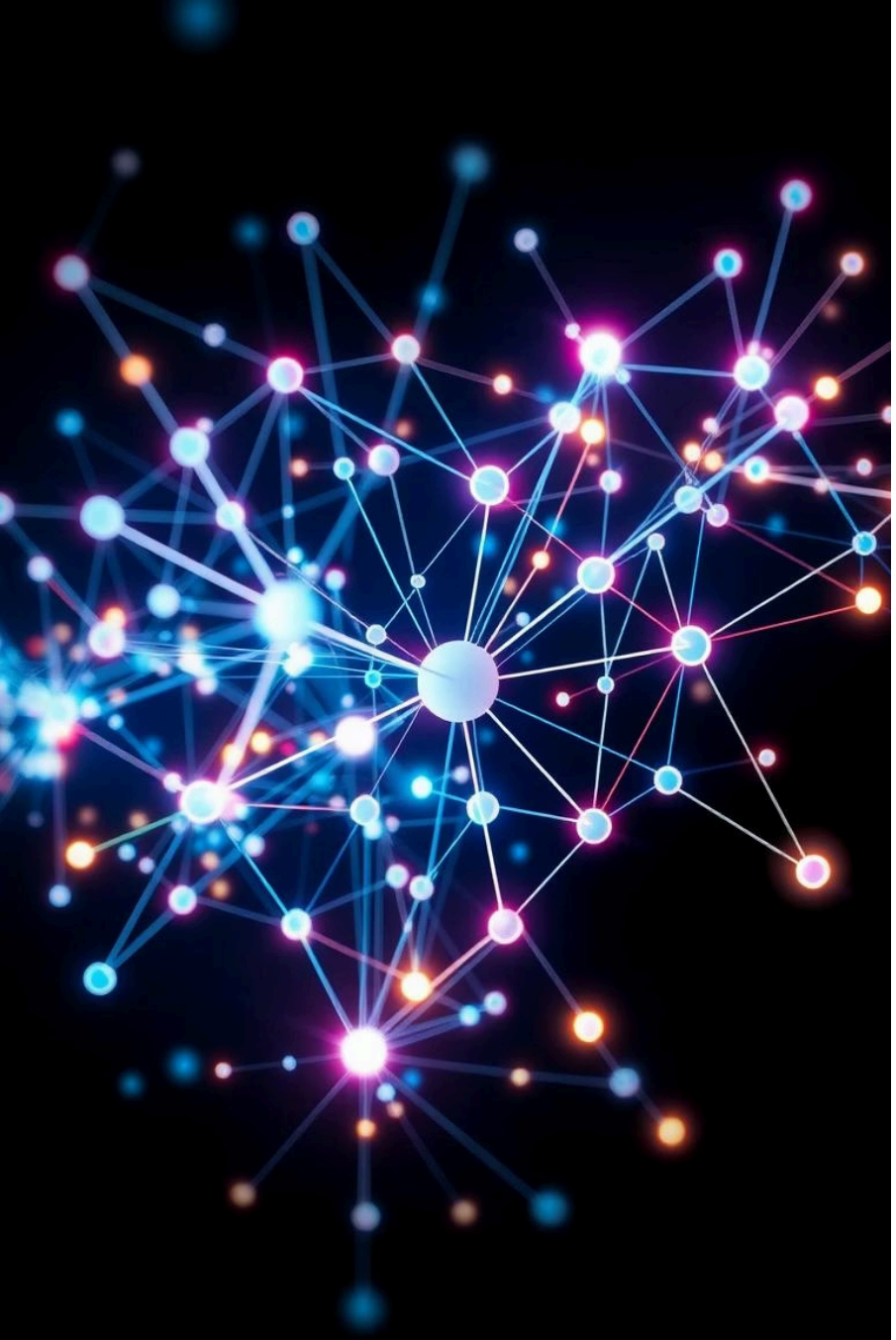


Benchmarking Community Detection for Graph-RAG Summarization

Student: Aviv Shimoni

Supervisor: Dr. Uri Itai

What is RAG?

Retrieval-Augmented Generation (RAG) improves LLMs by integrating external knowledge for better accuracy and adaptability.

Traditional LLMs

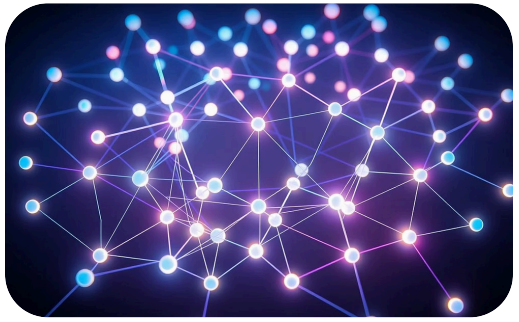
- Limited by static training data.
- Struggle with current or domain-specific info.
- Higher risk of hallucination when context is missing.

RAG Benefits

- Accesses relevant knowledge dynamically.
- Reduces expensive retraining.
- Lowers the likelihood of hallucinations.

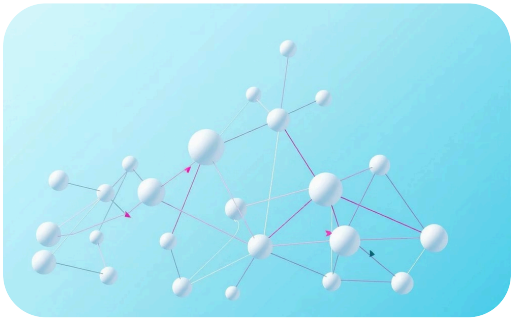
What is a graph?

A graph is a data structure made of **nodes (vertices)** connected by **edges**, representing relationships between entities.



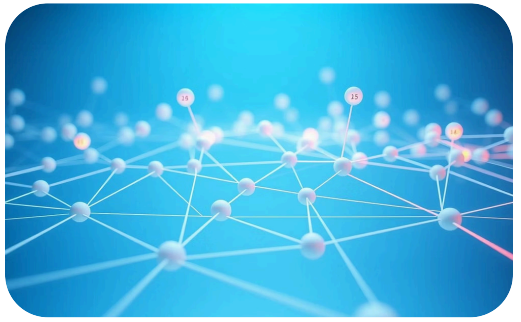
Undirected Edges

Two-way connections without a specific direction.



Directed Edges

One-way connections, flowing in a specific direction.



Weighted Edges

Edges with numerical values representing strength or cost.

Unweighted Edges

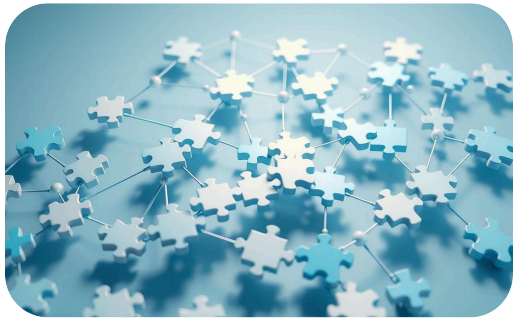
Simple connections without additional numerical values.

Why use a graph?

Graphs model complex connections, solving diverse problems efficiently.

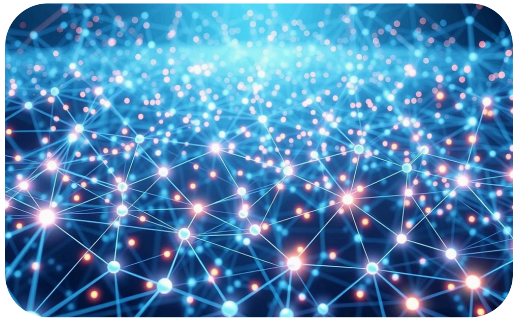
Structures Real-World Data

Organizes complex data into clear, structured graphs.



Solves Diverse Problems

Converts tasks into efficiently solvable graph problems.



Scales Efficiently

Manages millions of nodes and edges with integrity.

Powers Key Applications

Enables recommendations, GPS, search, and bio-network analysis.

From Local to Global: A Graph RAG Approach to Query-Focused Summarization (QFS)

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, Jonathan Larson

The Challenge

Summarizing large datasets for critical decision-making.

QFS Limitations

QFS struggles to scale and synthesize global insights from vast information.

RAG Limitations

RAG often falls short in answering broad global questions.

The Need

A scalable, context-aware solution is essential for robust global summarization.

Graph RAG

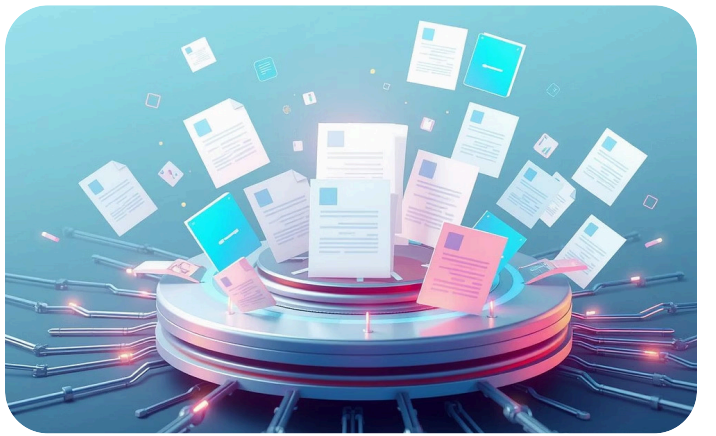
Graph RAG merges RAG with graph-based reasoning, enabling global summarization via hierarchical communities.

Scalability

Efficiently handles large datasets.

Comprehensive Insights

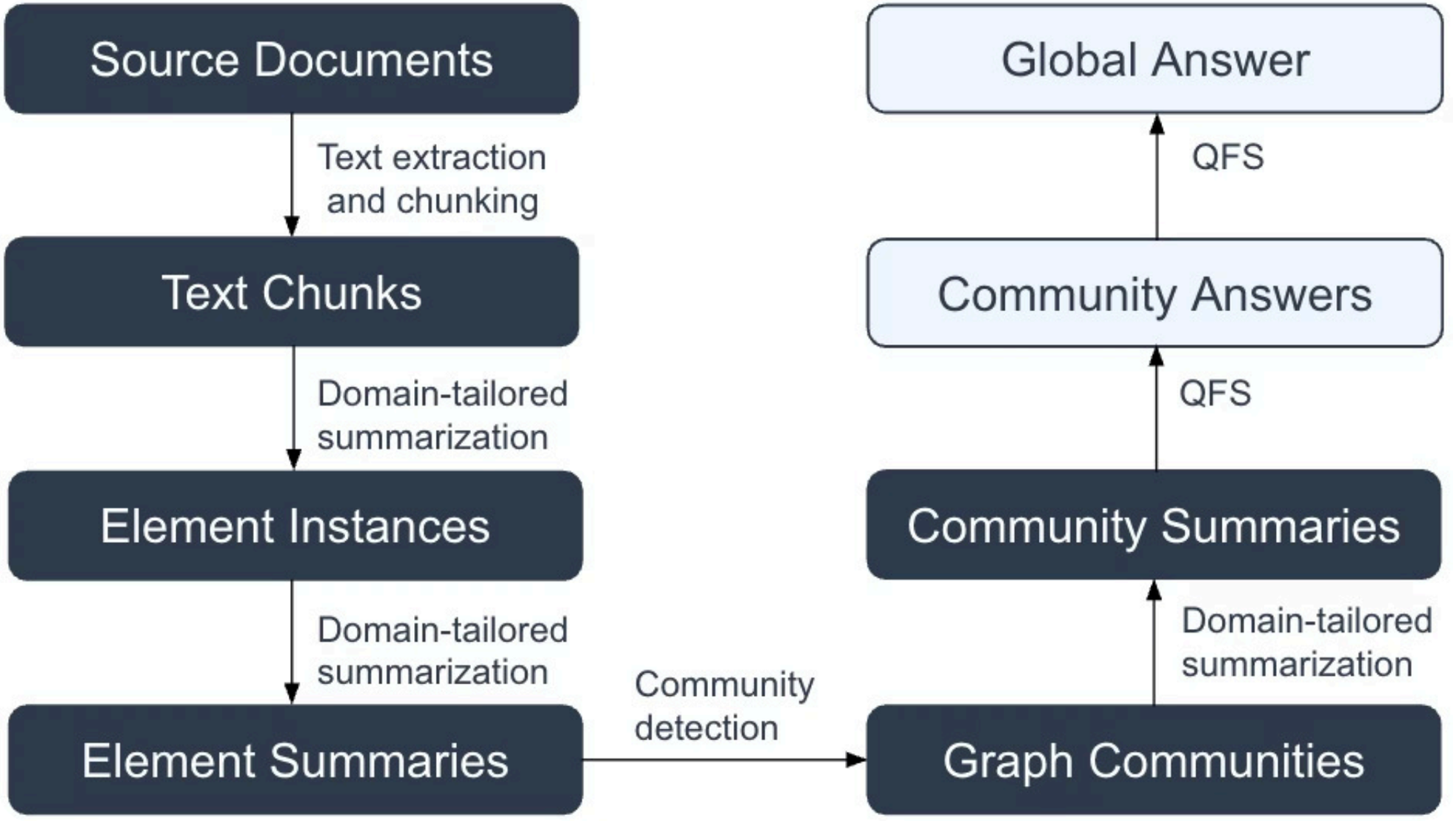
Integrates local and global insights.



Diverse Summarization

Generates adaptable, varied summaries.

Pipeline Design: How Graph RAG Works?



Benchmarking Community-Detection Algorithms

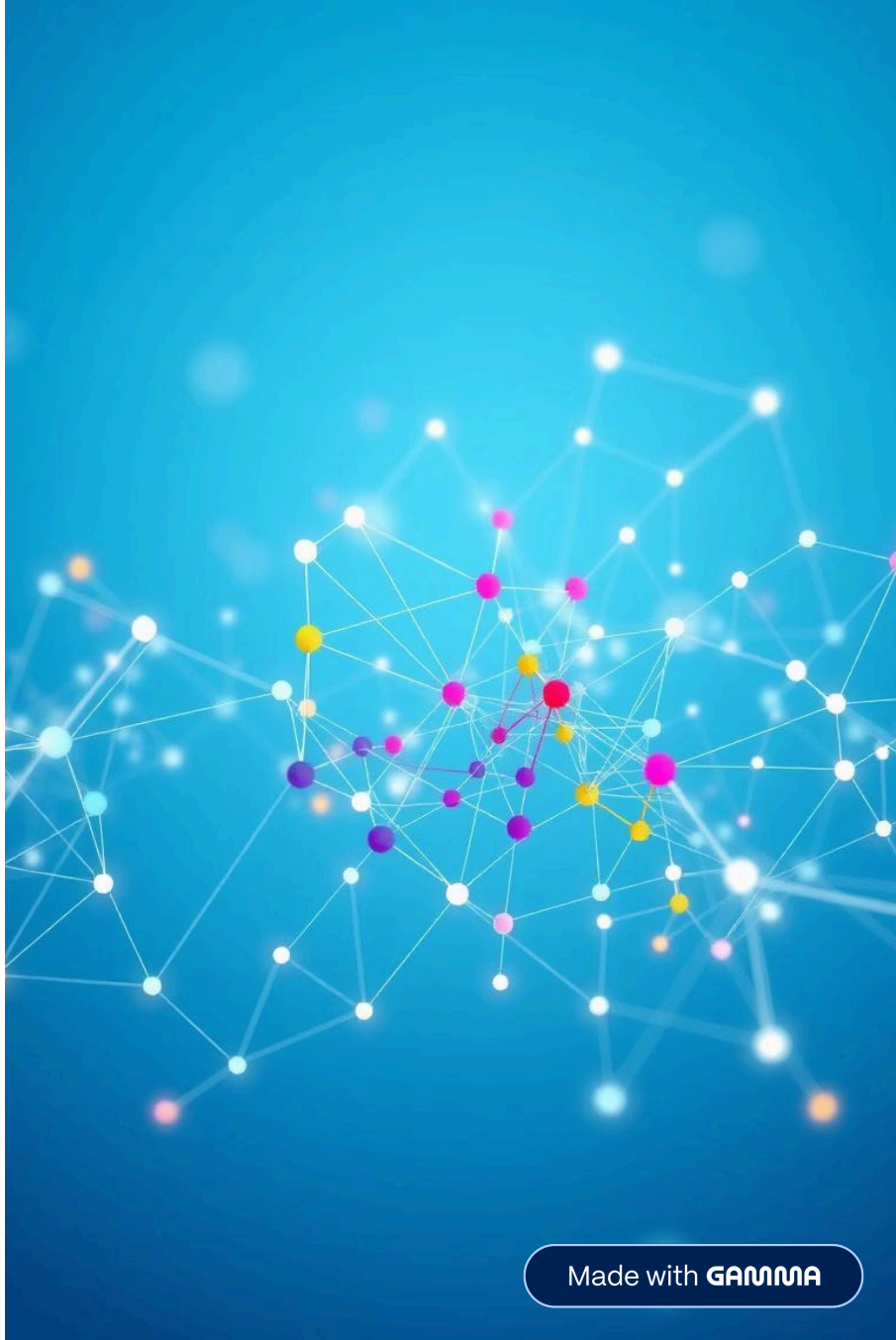
We evaluate multiple community detection algorithms beyond the single-algorithm baseline (Leiden) to find the best fit for Graph RAG.

Motivation

- Match algorithms to specific corpus/query types.
- Benchmark key metrics to reveal trade-offs
- Move beyond the single-algorithm baseline (Leiden)

Goal

- Provide guidance for selecting the optimal algorithm.



Why Graph Communities Matter

Segmenting a knowledge graph into coherent communities offers key benefits for query-focused summarization (QFS).

Topical Coherence

Minimizes topic drift, reducing hallucination risk.

Parallelism

Enables concurrent summarization across clusters, reducing latency.

Diversity Guarantees

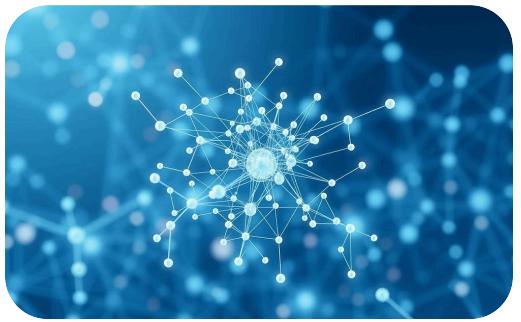
Ensures coverage of niche themes via per-community answers.

Community Detection Algorithms Benchmarked

We benchmarked seven popular algorithms chosen for diversity and graph analysis library presence.

Fiedler

K-Means

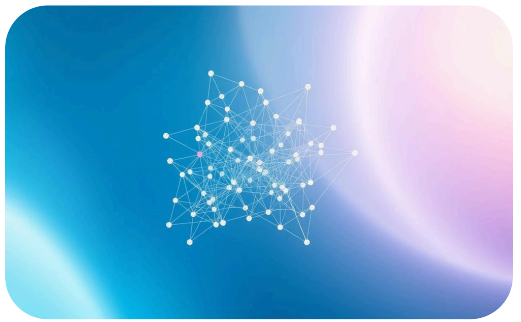


Leiden

Hierarchical

Weighted-Leiden

Girvan–Newman



Spectral

Evaluation Metrics

To assess community quality, we use nine intrinsic metrics grouped into three families.

Graph-theoretic	Modularity, Conductance, Coverage, Performance, Normalized Cut	Quantify structural cohesion and embedding-space separation.
Embedding	Silhouette coefficient, Calinski–Harabasz	Assess cluster separation and density in embedding space.
Distance	Intra-pairwise, Intra-centroid, Across	Measure within-community cohesion and between-community separation.

Project Implementation: Graph-RAG Pipeline

Our system converts raw text into a community-partitioned knowledge graph ready for query-focused analysis.

1

1. Document Preprocessing

Datasets split into 250-character chunks with overlap.

2

2. Entity–Relation Extraction

Llama 3.1 extracts triples (head, relation, tail) and Document →Entity edges.

3

3. Graph Storage & Indexing

Triples and node embeddings ingested into Neo4j with full-text index.

4

4. Community Detection

Partition the graph into cohesive communities to reveal its underlying structure and relationships.

Tech Stack

Python

Neo4j

stores the knowledge graph

Docker

containerizes Neo4j

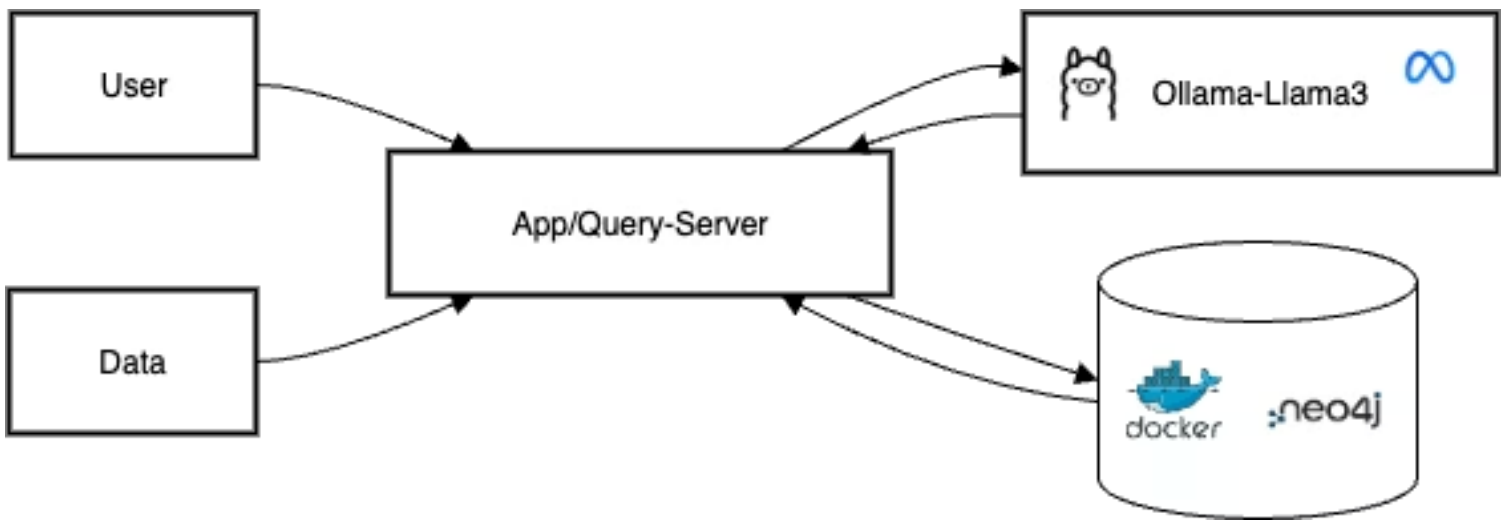
Ollama + Llama 3 8B

local LLM for entity–relation extraction

LangChain

orchestrates chunking, extraction, and graph writes

Project Implementation: Graph-RAG Pipeline



Datasets

We chose two compact, stylistically distinct public-domain novellas to stress-test graph topology:

Alice in Wonderland (□27 k words) – a fantasy story set in a surreal, dreamlike world

The Strange Case of Dr. Jekyll & Mr. Hyde (□29 k words) – a gothic tale exploring identity and duality

Adjacency Matrix: Alice vs. Jekyll

Alice in Wonderland

- **1,379 nodes**, **1,940 edges** → ultra-sparse (0.10% density)
- **Mean out-degree** ≈ 1.41
- Binary, unweighted links reflect episodic character interactions

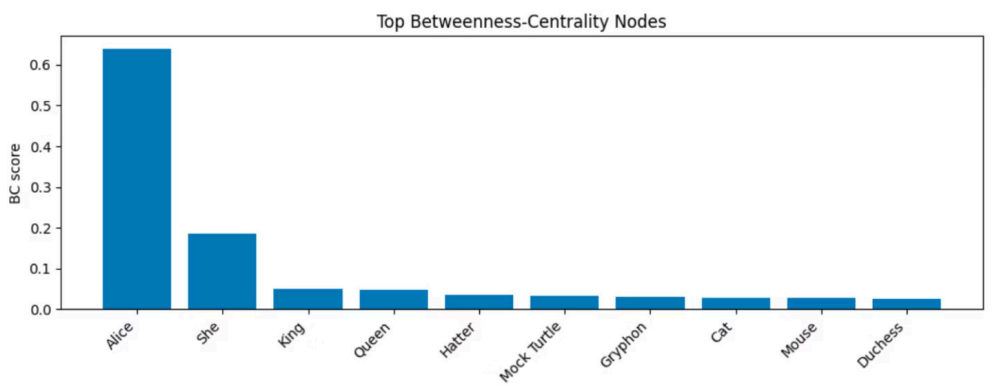
Dr. Jekyll & Mr. Hyde

- **1,602 nodes**, **2,343 edges** → similarly sparse (0.09% density)
- **Mean out-degree** ≈ 1.46
- Slightly denser core, with stronger **embedding structure**

Betweenness Centrality: Alice vs. Jekyll

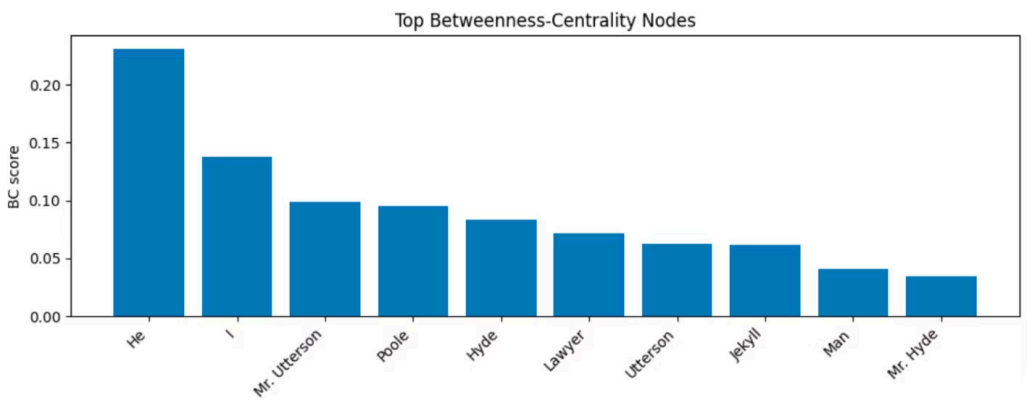
Alice in Wonderland

- Dominated by **Alice** (0.6384); huge drop after top node
- Max/mean ≈ 278 → highly centralized
- Interpretation: Alice connects most events; other characters are episodic



Dr. Jekyll & Mr. Hyde

- Led by **pronouns** (*He*, *I*) and **Mr. Utterson**
- Max/mean ≈ 165 → softer centralization
- Interpretation: Story flows through narration and testimony, not direct character interaction



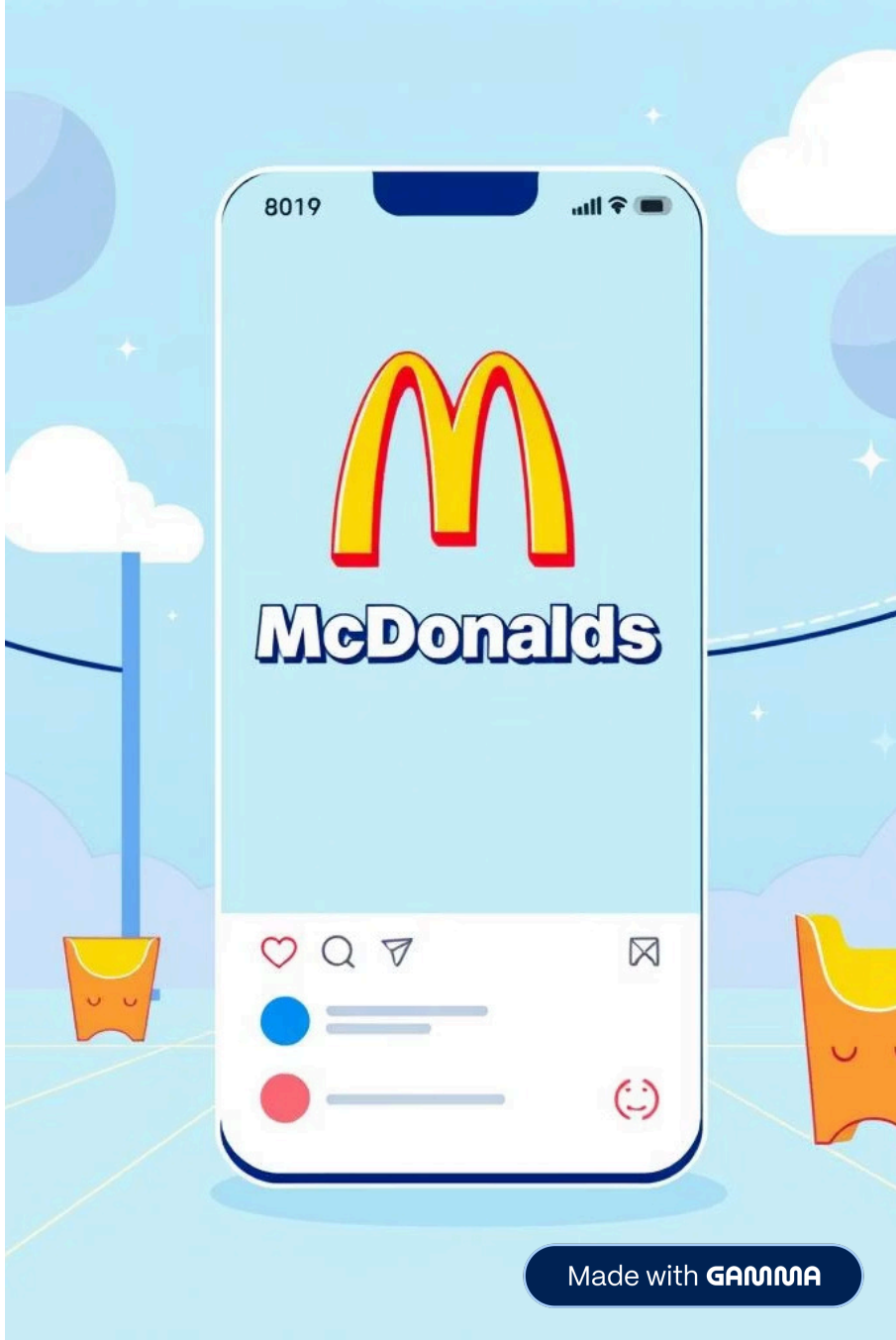
Hallucination tests

Alice in Wonderland: McDonald's Test

Prompt: “What did Alice eat at McDonald’s in Wonderland?”
Response: “There is no mention of Alice eating anything at McDonald’s in Wonderland.”

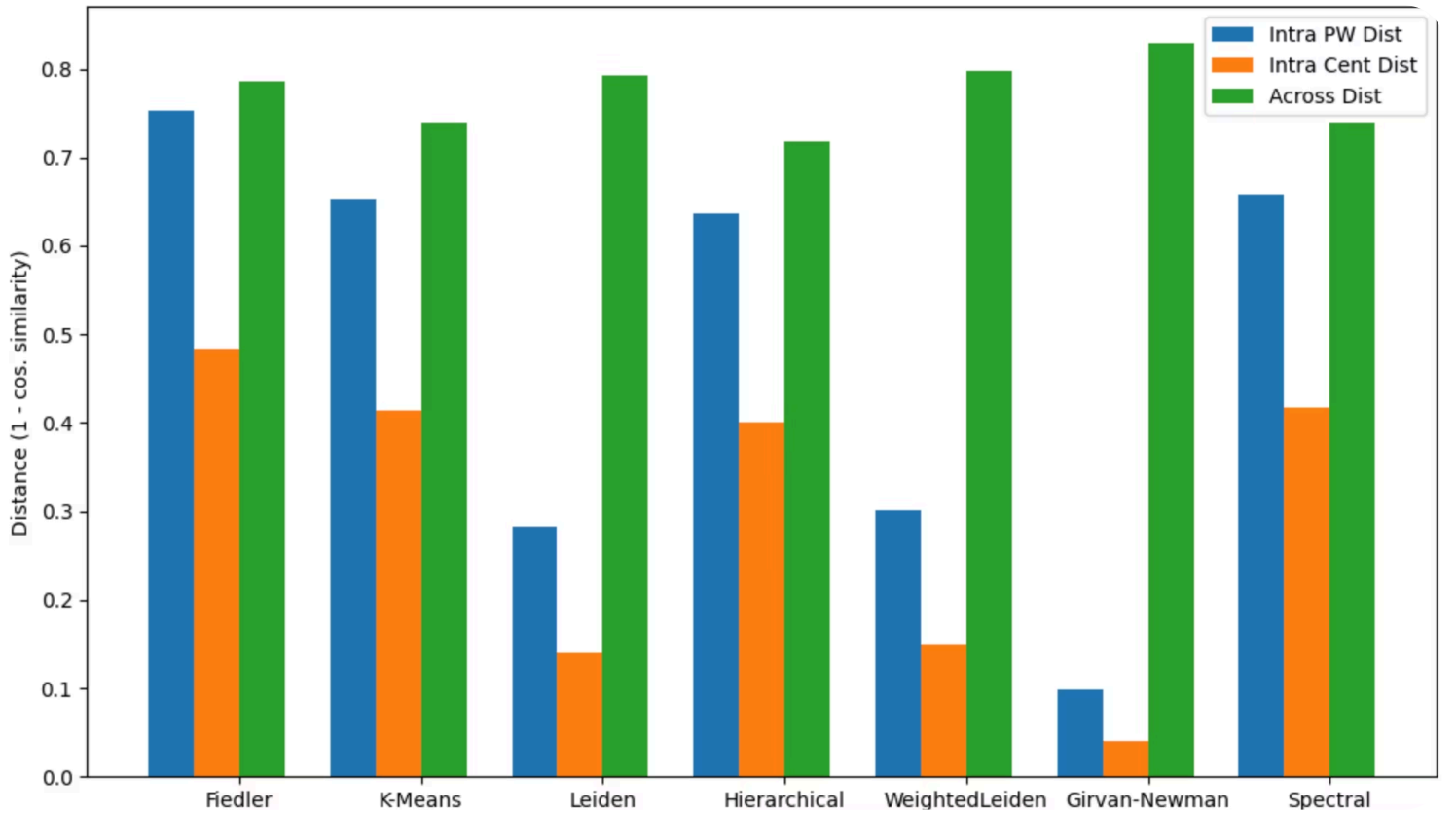
Dr. Jekyll & Mr. Hyde: Social Media Test

Prompt: “What social media platform did Dr. Jekyll use to document his experiments?”
Response: “There is not enough information in the provided context to answer this question.”

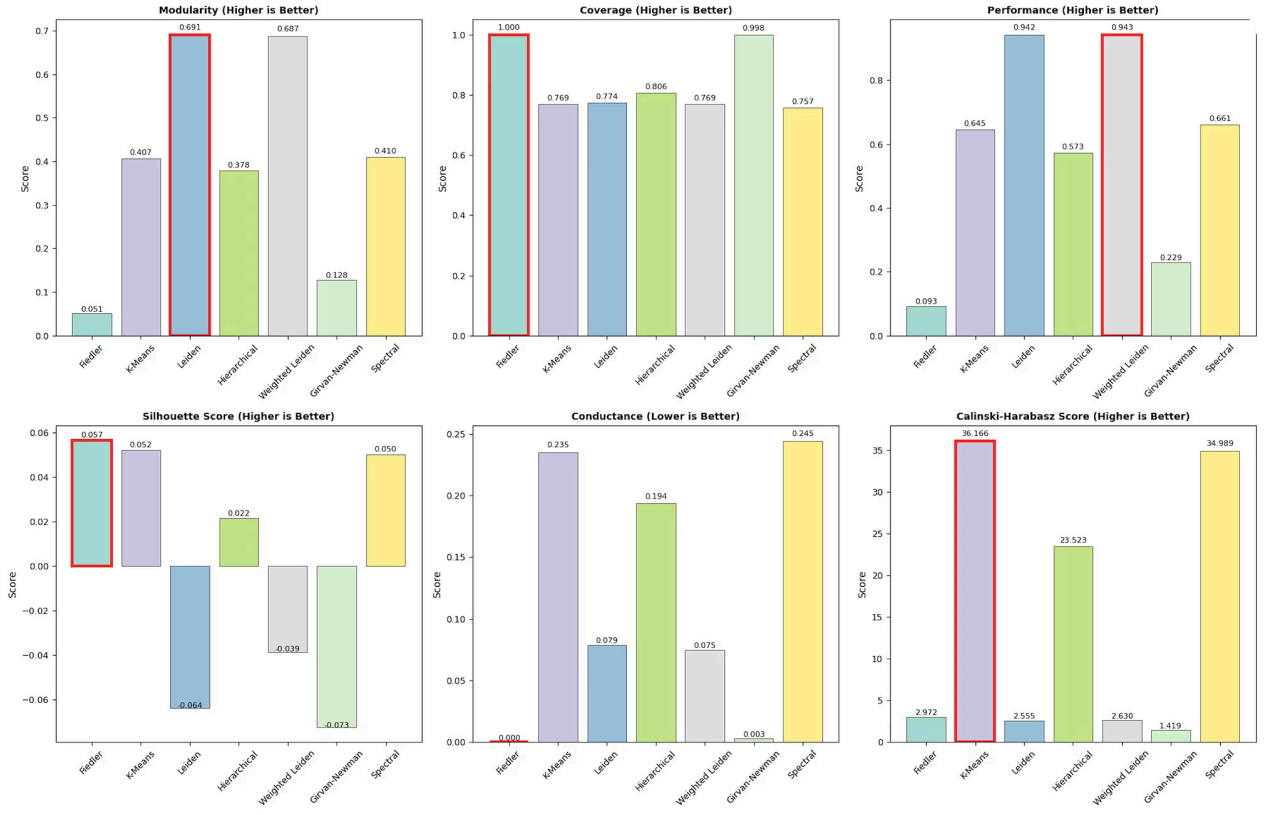


Alice in Wonderland: distance metric comparison

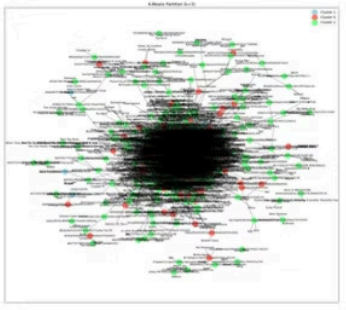
Distance metrics (Lower = Better for Intra, Higher = Better for Across)



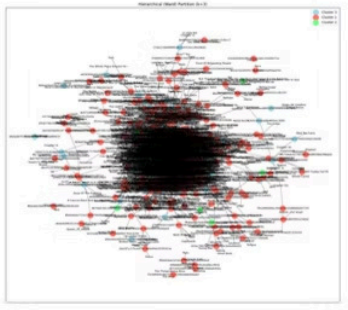
Alice in Wonderland: graph-based metrics



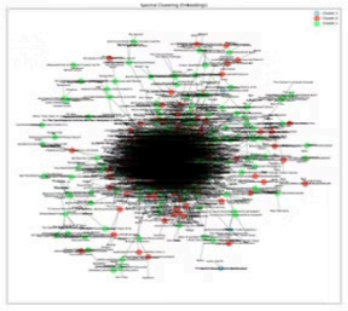
Alice in Wonderland: Graph



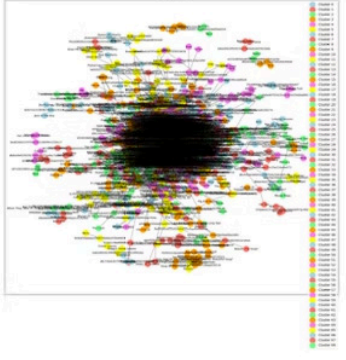
(a) K-Means



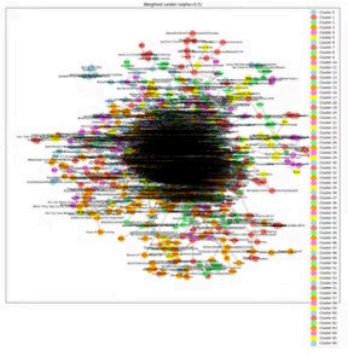
(b) Hierarchical



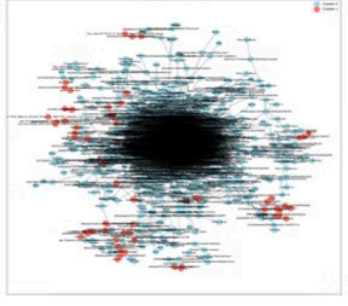
(c) Spectral



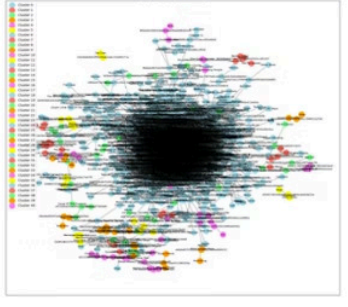
(d) Leiden



(e) Weighted-Leiden



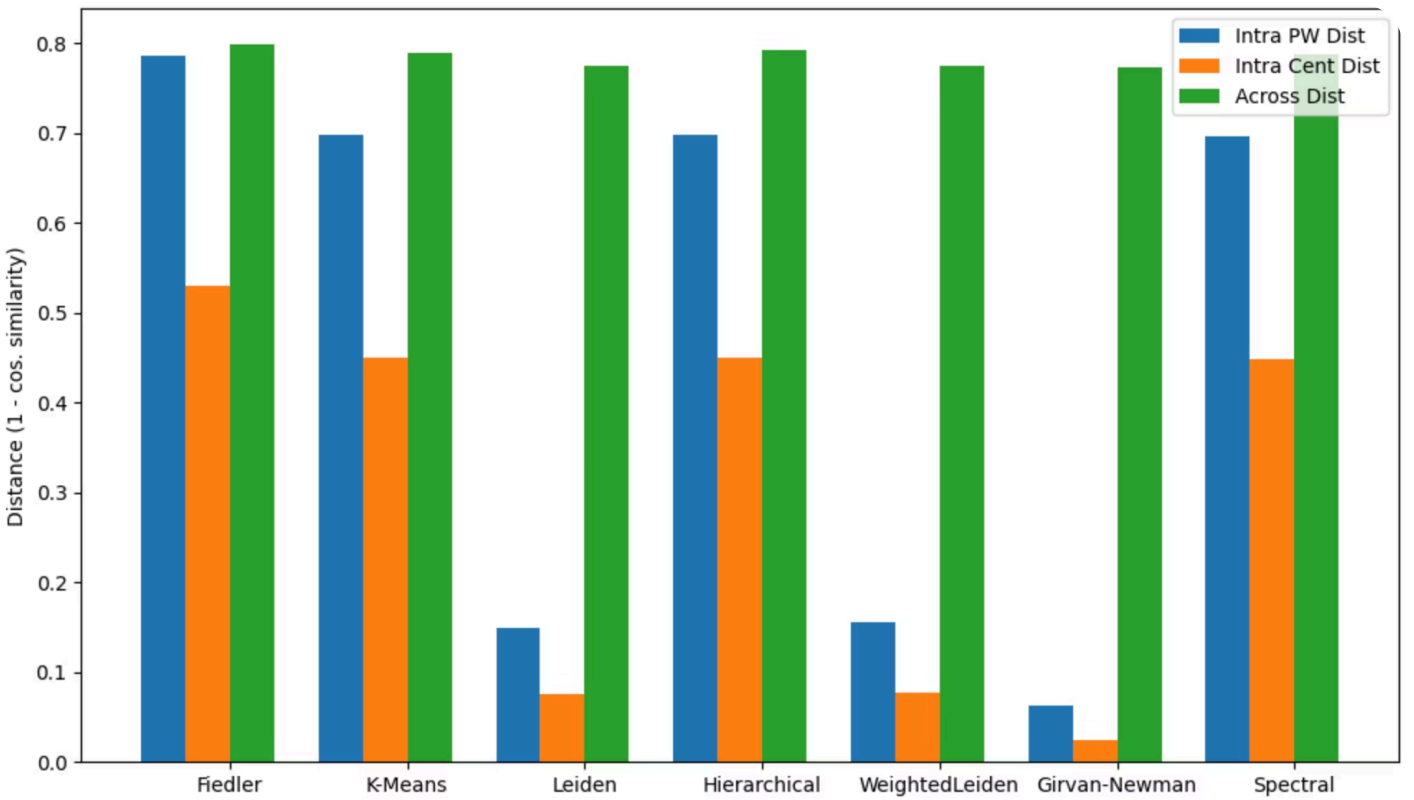
(f) Fiedler



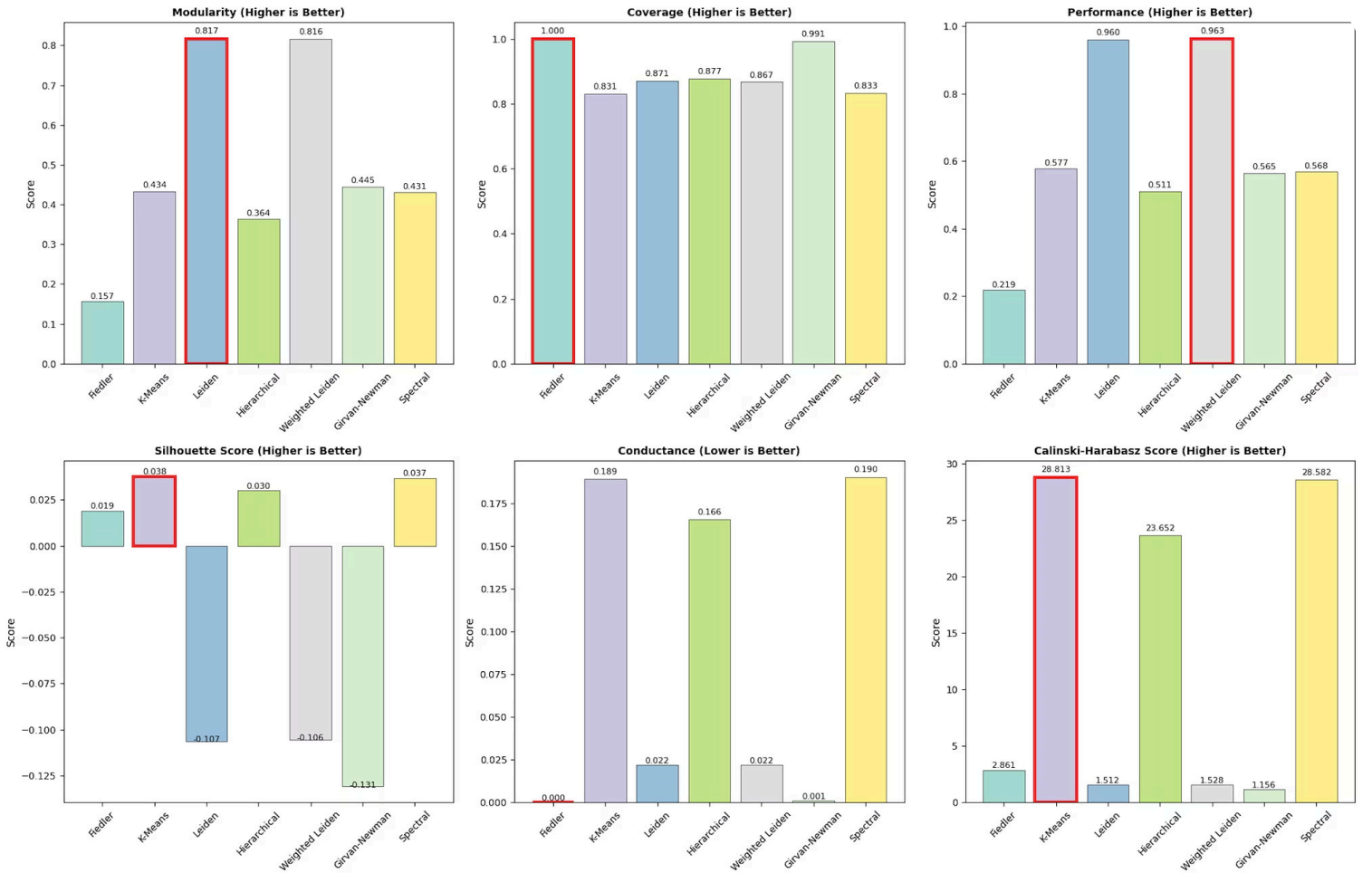
(g) Girvan–Newman

Dr. Jekyll & Mr. Hyde: distance metric comparison

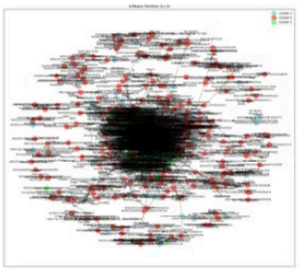
Distance metrics (Lower = Better for Intra, Higher = Better for Across)



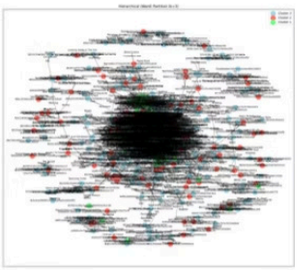
Dr. Jekyll & Mr. Hyde: graph-based metrics



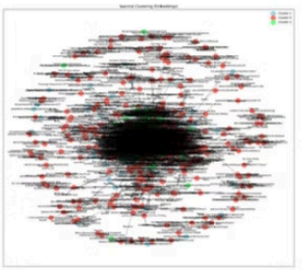
Dr. Jekyll & Mr. Hyde: Graph



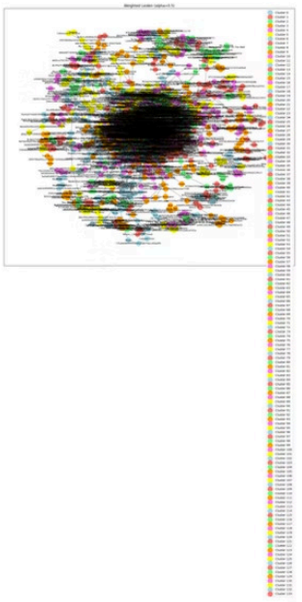
(a) K-Means



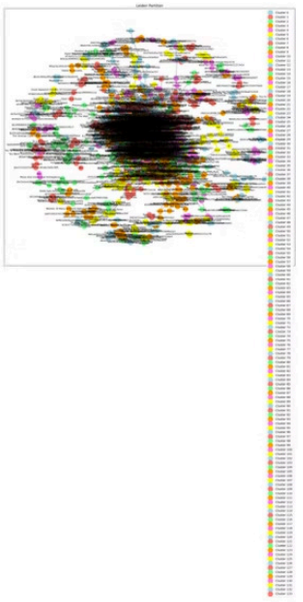
(b) Hierarchical



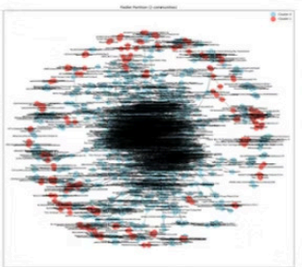
(c) Spectral



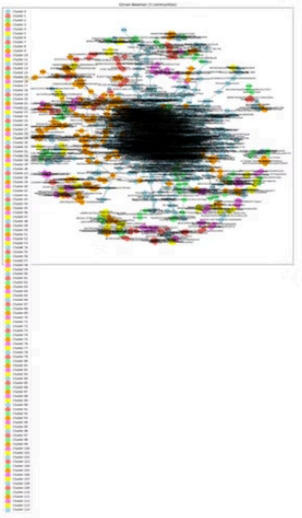
(d) Leiden



(e) Weighted-Leiden

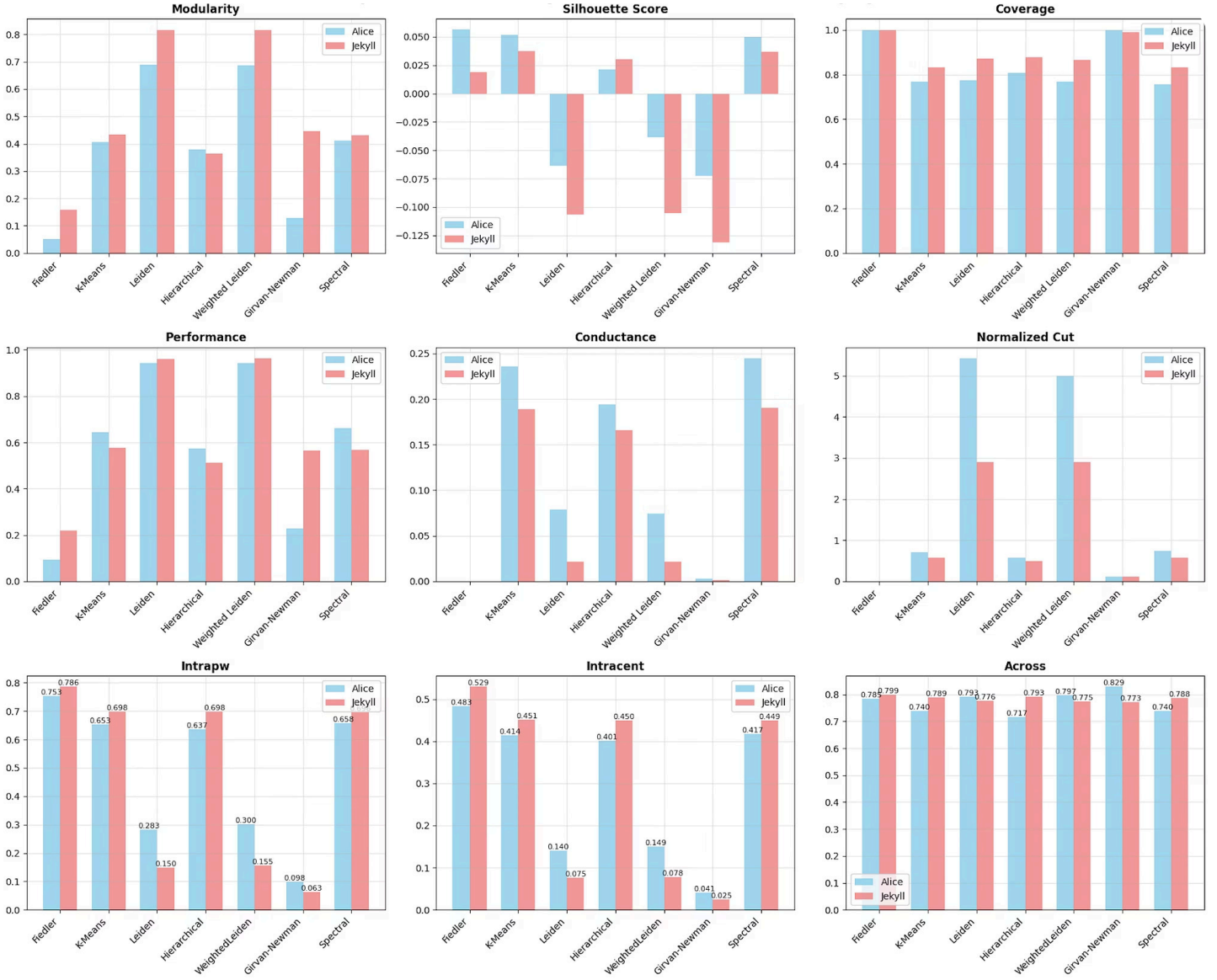


(f) Fiedler



(g) Girvan–Newman

Cross-Dataset Performance - Alice vs Jekyll



Key Findings

Weighted Leiden

- Strong cross-domain robustness with a balanced composite score
- Achieved a −16% reduction in Intra-Pairwise Distance

Leiden

- The "modularity champion" on both datasets
- Modularity values ranged from 0.69 (Alice) to 0.82 (Jekyll)

Fiedler

- Demonstrated perfect coverage (1.00)
- Delivered stable separation in the Across metric (~0.72–0.75)

Girvan–Newman

- Best semantic cohesion scores on both datasets (0.15 and 0.10)
- Consistent high Across metric (0.78+ in both datasets)

Community-Detection Summary

Weighted Leiden

The safest all-rounder, consistently top-2 and nearly halves cohesion distance on Jekyll.

Leiden

The "modularity champion" for optimal structural partitioning.

Girvan–Newman

Produces the tightest and best-separated clusters.

Fiedler

Achieves perfect coverage and clean, coarse partitioning.

Excellence Criteria

Algorithms meeting tight intra-cluster cohesion (≤ 0.30) and high separation (≥ 0.65).

Practical Recipe

Start with Leiden for balance, use Weighted Leiden for varied domains, and Girvan–Newman for ultra-compact clusters.

Future Work

Auto-selector Pipeline

Train a meta-model to pick the best algorithm per corpus/query in real time.

Dynamic Thresholds

Adapt “excellence” cut-offs based on corpus size and topic diversity.

Hyper-tuning & Ensembling

Explore parameter sweeps and hybrid algorithm blends.

Streaming / Incremental Clustering

Extend methods to handle live content without full re-compute.